

TOPIC 39/60

What is KV Cache in LLMs?

Speeding Up Inference.

How storing past math computations prevents LLMs from slowing down on every new token generated.

Ashish Jangra

linkedin.com/in/ashish-jangra



THE PROBLEM

The Groundhog Day Loop

LLMs generate text sequentially, one single token at a time. To output token #100, standard Transformer attention requires analyzing its relationship to all 99 previous tokens.

Wasted Attention Math

Without a caching system, when generating token #101, the GPU is forced to recompute the Keys and Values for tokens 1 through 100 all over again.

Result: $O(N^2)$ quadratic calculations that make text generation slower and slower as your context window grows.

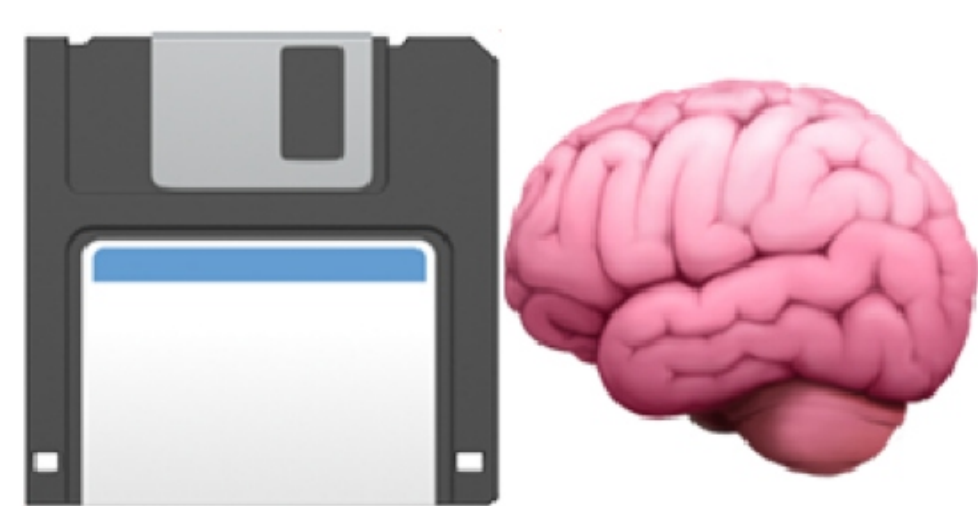
Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE CONCEPT

Saving the Past



What is KV Cache?

Instead of discarding the Key (\$K\$) and Value (\$V\$) matrices computed for preceding tokens, we store them directly in GPU memory (VRAM). This dynamic storage array is called the KV Cache.

Without KV Cache

Redundant matrix multiplications on every token step. Drags inference speeds to a crawl.

With KV Cache

Compute once, store in VRAM. Only project values for the newly generated token.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



UNDER THE HOOD

The Math Shortcut

How we reduce computation from a massive matrix sweep to a single vector append:

Step 1: Compute New Elements Only

For the newly generated token x_t , we project its Query, Key, and Value: q_t , k_t , v_t .

Step 2: Append & Dynamic Match

We append k_t and v_t to our rolling K_{cache} and V_{cache} . The single vector q_t is then multiplied against the entire K_{cache} to determine attention scores.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



PHASES OF INFERENCE

The Two Execution Modes

LLM text generation is split into two fundamentally different compute regimes:

1. Prefill Phase

Processes the entire prompt context in parallel. Calculates the initial K and V matrices to populate the Cache.

Compute-Bound

2. Decode Phase

Generates tokens sequentially. Reuses the KV Cache, computing only one new token projection at a time.

Memory-Bound

The transition to decoding makes inference highly bottlenecked by GPU memory transfer rates rather than raw compute FLOPS.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE DOWNSIDE

The Memory Cost

Speed isn't free. The KV Cache can expand rapidly, hogging massive pools of GPU VRAM. Its scale is determined by the formula:

$$\text{Size} = 2 \times L \times H \times D \times S \times B \times P$$

- L: Number of layers

H: Number of Key-Value heads

D: Head Dimension width

S: Sequence length (tokens)

B: Batch size

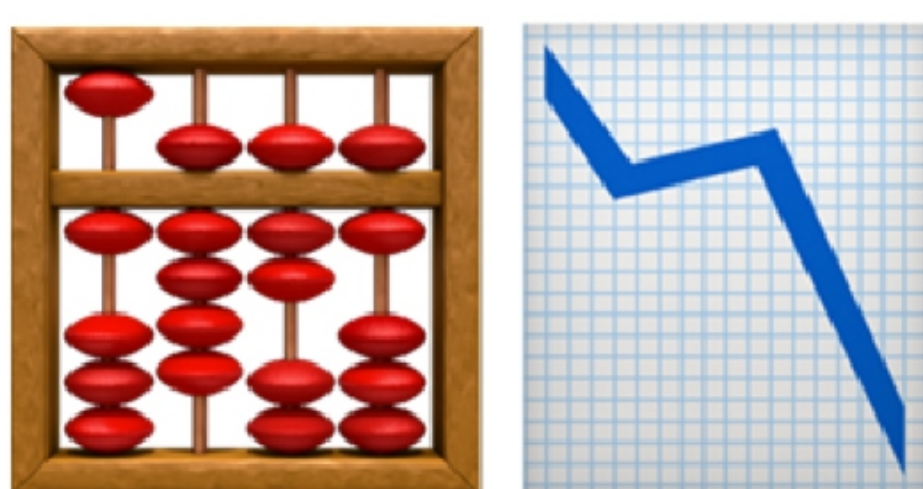
P: Bytes per param (e.g. FP16 = 2)

For long-context or multi-user generation, the KV cache can easily dwarf the actual size of the model's weights!



CASE STUDY

KV CACHE VRAM MATH



Let's calculate the memory required for a **Llama-3-8B** model running FP16 with a batch size of 16 and sequence length of 8,192 tokens:

Model specs: 32 layers, 8 Key-Value Heads, Head Dim 128

Formula: $2 \times 32 \times 8 \times 128 \times 8192 \times 16 \times 2$ bytes

Total Cache: ~17.18 Gigabytes of VRAM!

Warning: That's nearly as much memory as the static model weights themselves (16 GB)!

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



OPTIMIZATION STRATEGIES

Smarter Key-Value Allocation

How modern LLM designs structure Attention layers to shrink the memory footprint:

1. Multi-Head Attention (MHA)

Every query head has its own Key and Value head. Highly expressive but produces a massive, bloated KV cache.

2. Multi-Query Attention (MQA)

All query heads share a single Key and Value head. Drastically cuts KV Cache size but can hurt language capabilities.

3. Grouped-Query Attention (GQA) - The Gold Standard

Query heads are divided into groups (e.g. 4 or 8), with each group sharing one KV head. Used by Llama 3 to combine speed and quality.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE MEMORY REVOLUTION

vLLM & PagedAttention

Static Allocation Waste

Standard software reserves a contiguous block of VRAM for the maximum context length up front. Because actual responses are shorter, up to 60-80% of KV Cache VRAM is wasted.

⚡ PagedAttention System

Treats KV Cache memory like an OS virtual memory page table. KV blocks are stored in non-contiguous physical pages and mapped dynamically as tokens generate.

The Result

Eliminates memory fragmentation, reclaims 96% of wasted VRAM, and enables up to 4x throughput bottlenecks for production engines.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



KV Cache

CONCEPT UNLOCKED



Bypasses $O(N^2)$ quadratic calculations



Stores Key & Value tensors in VRAM



Powered by GQA to save massive memory footprint



Managed by PagedAttention to stop fragmentation

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

Ashish Jangra

linkedin.com/in/ashish-jangra

